

OWASP JUICE SHOP ASSIGNMENT

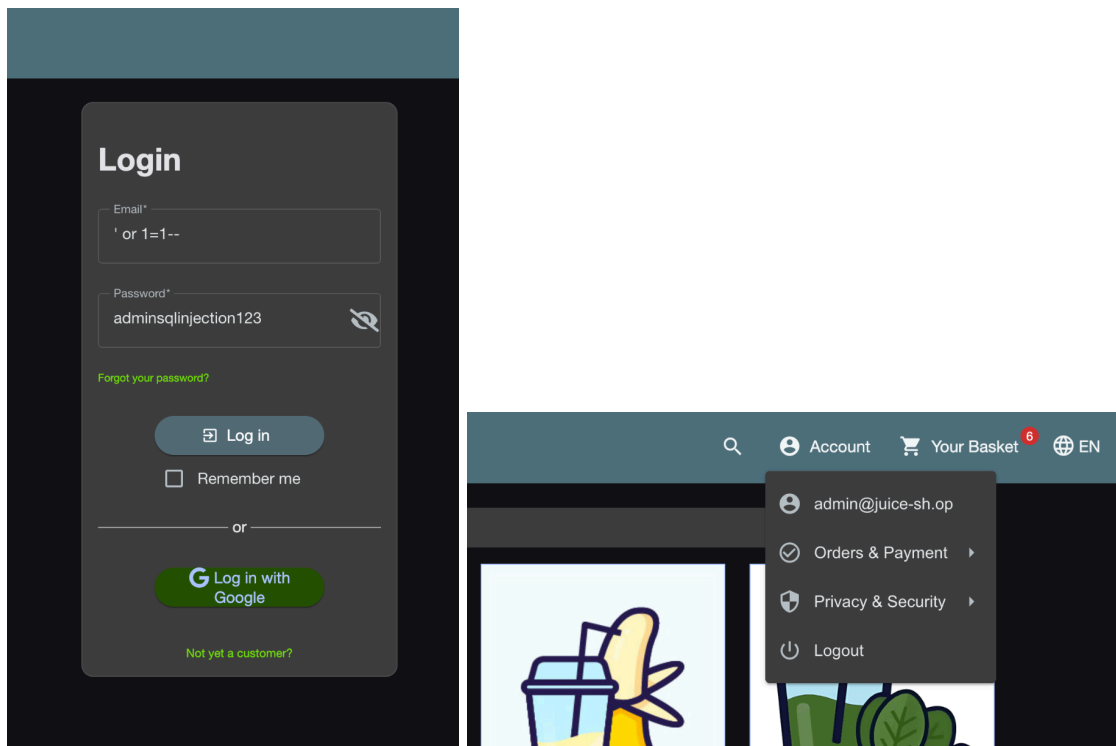
Name: Zaahir Sharma

UIN: 134004682

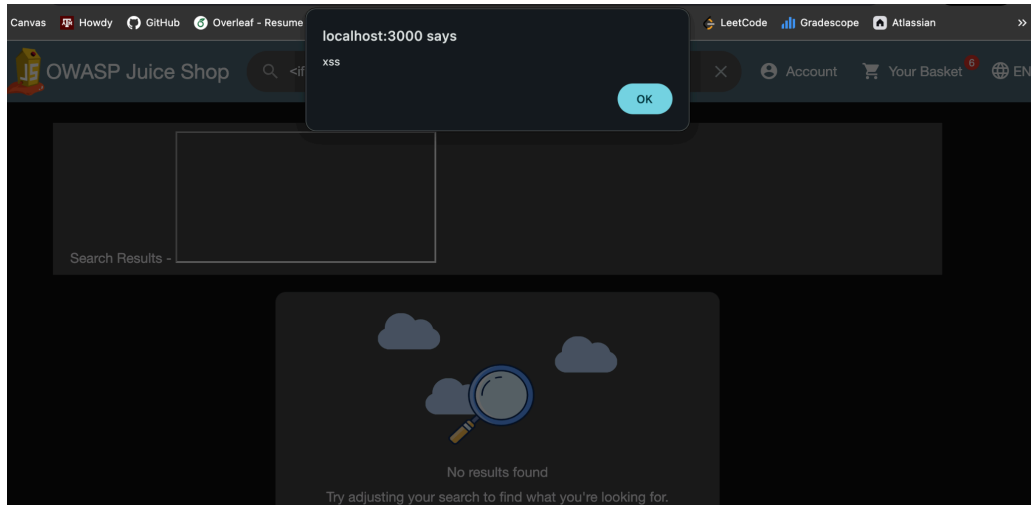
Date: 10/02/2026

PART 1 - Secure Feature Design

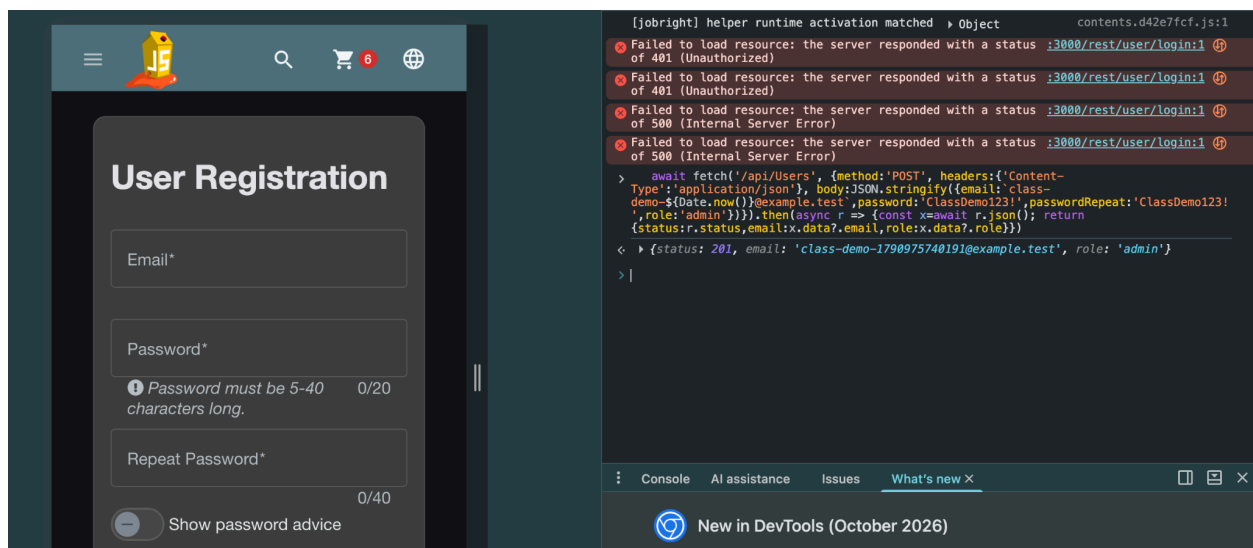
The Juice Shop evidence shows three vulnerabilities: SQL injection bypassed login and displayed the administrator identity; a search injection produced an “xss” browser alert; and registration accepted a client-supplied administrator role, returning HTTP 201. A secure registration system should use parameterized queries so input cannot change SQL, and render untrusted text using `textContent` so it cannot execute as HTML. Registration should allowlist permitted fields and assign customer roles server-side, with authorization checks on protected endpoints. Passwords should be stored as salted hashes, for example await `bcrypt.hash(password, 12)`, and verified using `bcrypt.compare`. These measures address the observed injection and privilege-escalation risks.



SQL injection: The login input and subsequent account menu show the administrator identity after the injection attempt.



XSS: Juice Shop displays an “xss” browser alert during the search injection test.



Registration: The registration response shows HTTP 201 and the returned role “admin”.

PART 2 - Front-end Form Implementation

I built a Juice Shop-inspired login form using HTML, CSS, and JavaScript with a Node.js backend. The form includes email and password fields, a password visibility toggle, and validation feedback. `validateLogin` rejects empty inputs, emails without “@”, and passwords shorter than eight characters. The server independently repeats these checks; a direct invalid request returned HTTP 400. SQLite prepared statements protect account lookup, bcrypt verifies passwords, and successful authentication creates a short-lived `HttpOnly` session. Submitted email is displayed using `textContent`. The README explains installation, demo credentials, and testing. The screenshots show successful login and validation rejection.

Public GitHub repository: <https://github.com/zaahirsharma/juice-shop-login-form>

Two side-by-side screenshots of a login form. The left screenshot shows the form with demo credentials (student@example.com, Student123!) entered. The right screenshot shows the form after a successful login, displaying "Login successful." and "Submitted email: student@example.com".

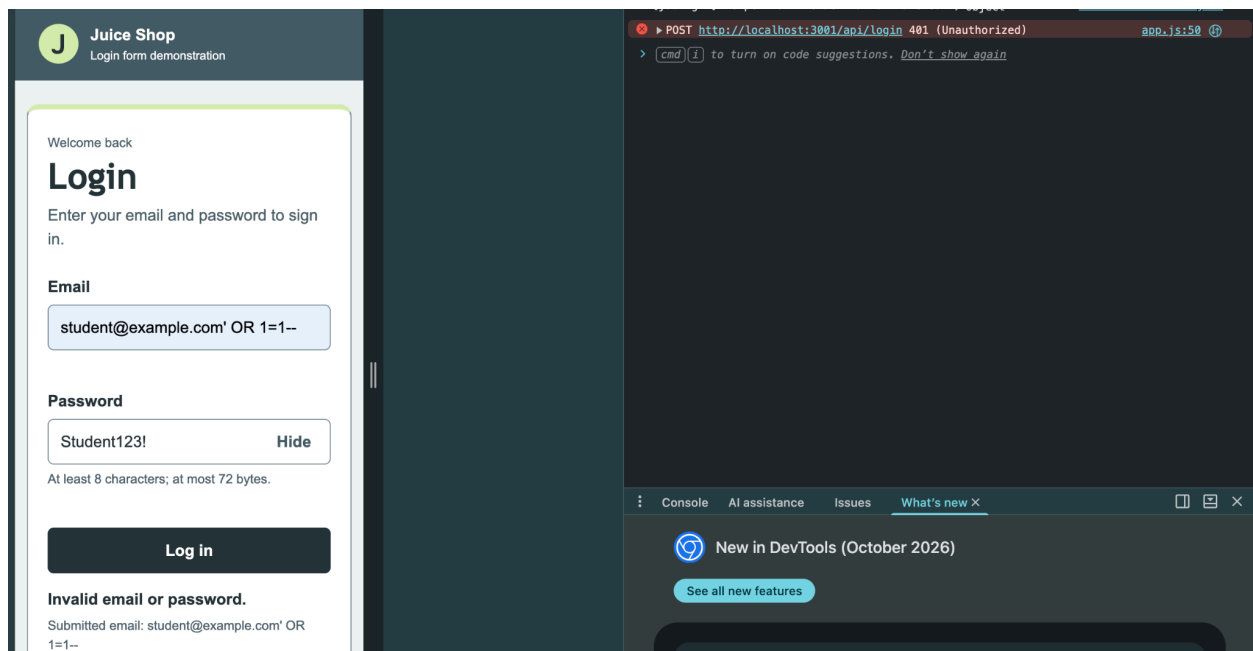
Login: The demo credentials produce “Login successful.”.

A screenshot of the login form with empty fields, showing validation errors: "Enter your email." and "Enter your password.". To the right, a DevTools console shows a failed API request: "POST http://localhost:3001/api/login 400 (Bad Request)".

Validation: Empty browser submissions show field errors; a separate direct API request with invalid values returns HTTP 400.

PART 3 - Exploitation Attempt on my Form

On my local login form, I submitted `student@example.com' OR 1=1--` with `Student123!` and clicked Log in. Login was rejected and the saved API results recorded HTTP 401. Next, I submitted `<img src=x onerror=alert("xss")>@example.com` with the same password. The form displayed the input literally and rejected login, with no alert observed. Both inputs passed the basic client validation, allowing the server and rendering defenses to be tested. Prepared statements kept the SQL input separate from query syntax. For an unsafe rendering implementation, replacing `innerHTML` with `textContent` would prevent this DOM injection. These failed attempts demonstrate protection against the tested inputs, not every possible vulnerability.



SQL injection: The form rejects the submitted SQL injection input with “Invalid email or password.”

Welcome back

Login

Enter your email and password to sign in.

Email

Password

 Hide

At least 8 characters; at most 72 bytes.

Log in

Invalid email or password.

Submitted email: <img src=x
onerror=alert('xss')>@example.com

Try the demo account

Email: student@example.com
Password: Student123!

Use these sample credentials for this local exercise.

XSS: The submitted markup appears as literal text, and login is rejected.

SOURCES

OWASP SQL Injection Prevention:

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Cross Site Scripting Prevention:

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Password Storage:

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Mass Assignment:

https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html